PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Consumo y Documentación de una API con React

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Nicolas Ruiz Colorado**

Ficha: 3312932

Competencia:

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **24** de **octubre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Consumo y Documentación de una API con React 5

1. Creación del proyecto Frontend 5

1.1. Creación vite React 5

1.2. Configuración de entorno 6

1.3. Creación de js con dirección URL de la API 7

1.4. Código Frontend 7

1.5. Evidencias 11

2. Creación proyecto backend 12

2.1. Creación de carpeta y abrirla con vscode 12

2.2. Configuración del entorno 13

2.3. Código backend 14

3. Investigación sobre consumo de API 19

3.1. ¿Qué es una API y para que sirve en una aplicación web? 19

3.2. ¿Qué diferencia existe entre frontend y backend? 20

3.3. ¿Qué significa consumir una API? 20

3.4. ¿Qué función cumple fetch()? 20

3.5. ¿Por qué se utilizan async y await en una petición HTTP? 20

3.6. ¿Qué es una promesa en JavaScript? 21

3.7. ¿Para que sirve response.json()? 21

3.8. ¿Qué diferencia existe entre una petición GET y una petición POST? 21

3.9. ¿Qué función cumple las cabezeras HTTP y por qué se utiliza Content-Type:application/json? 22

3.10. ¿Por qué se utiliza JSON.stringify() al enviar datos en una petición POST? 22

3.11. ¿Qué diferencia existe entre un error de conexión y una respuesta HTTP con código 400? 22

3.12. ¿Qué función cumplen useState y los eventos onClick y onSubmit? ... 23

3.13. ¿Qué es CORS y por qué puede afectar la comunicación entre React y Express? 23

3.14. ¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend? 24

3.15. ¿Por qué no se debe considerar exitosa toda petición solamente porque fetch() devolvió una respuesta? 24

4. Evidencias de pruebas 25

4.1. Consultar /inicio 25

4.2. Consultar /información 26

4.3. Enviar un formulario completo 27

4.4. Enviar campos vacíos 28

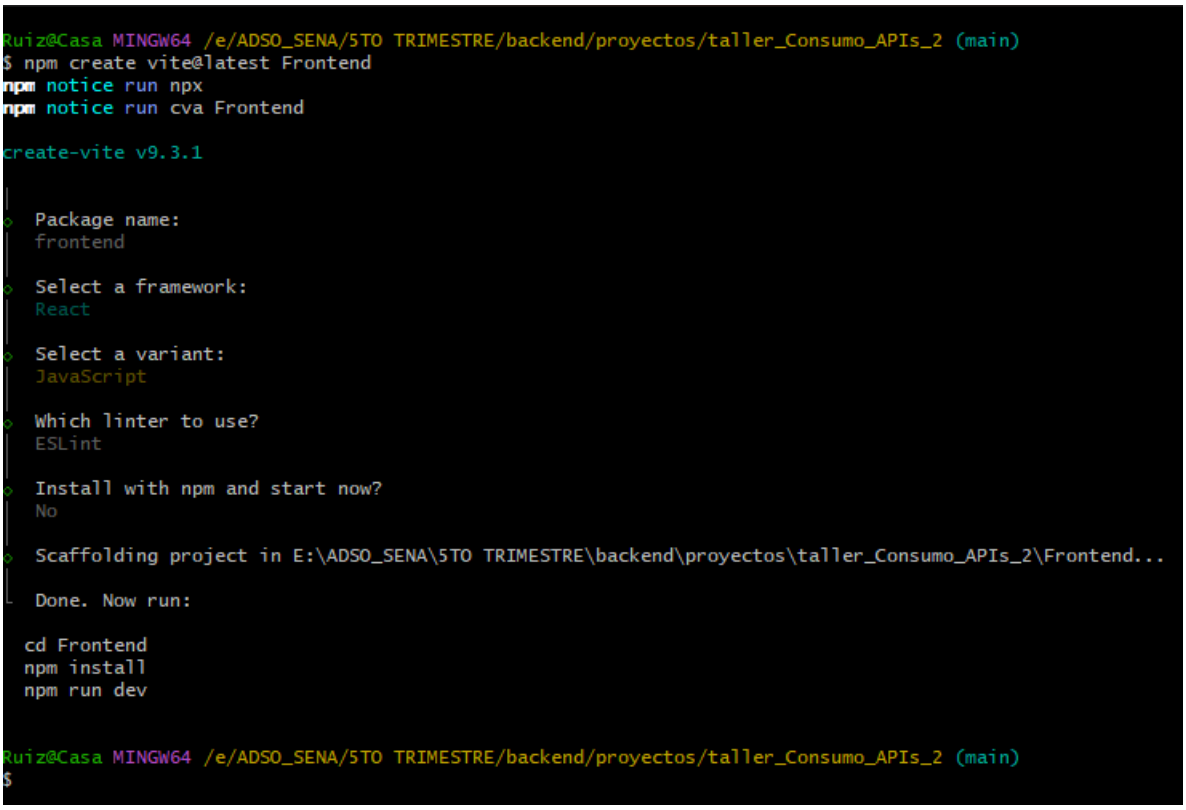
4.5. Enviar un correo invalido 28

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Consumo y Documentación de una API con React

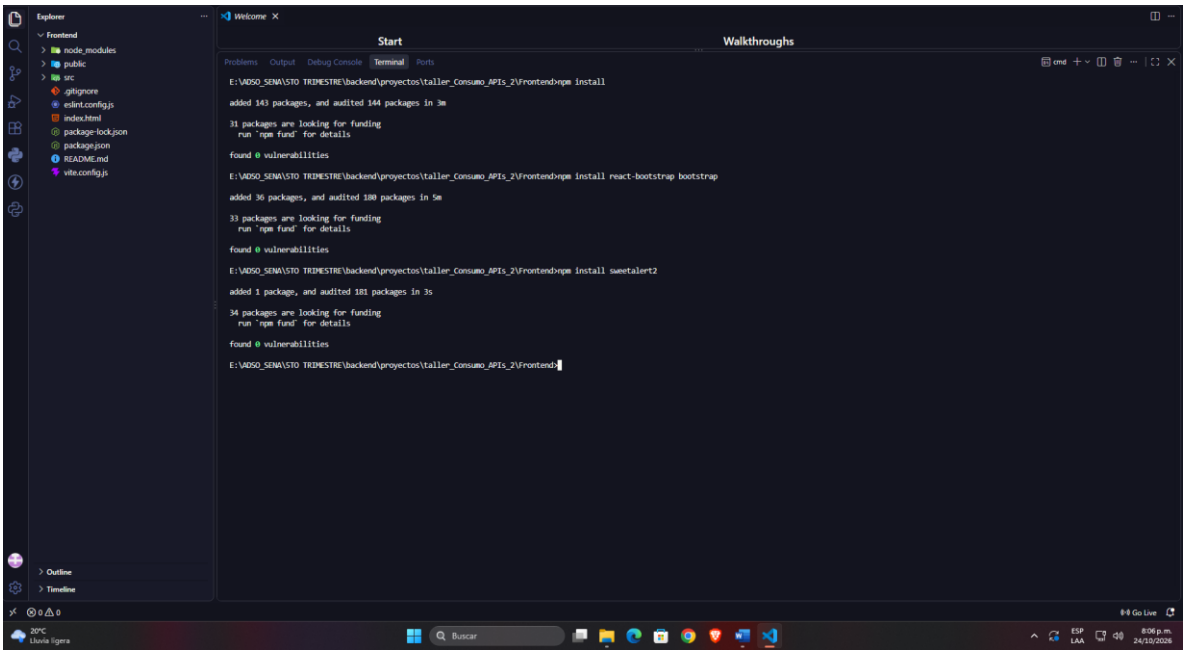
1. Creación del proyecto Frontend

1.1. Creación vite React

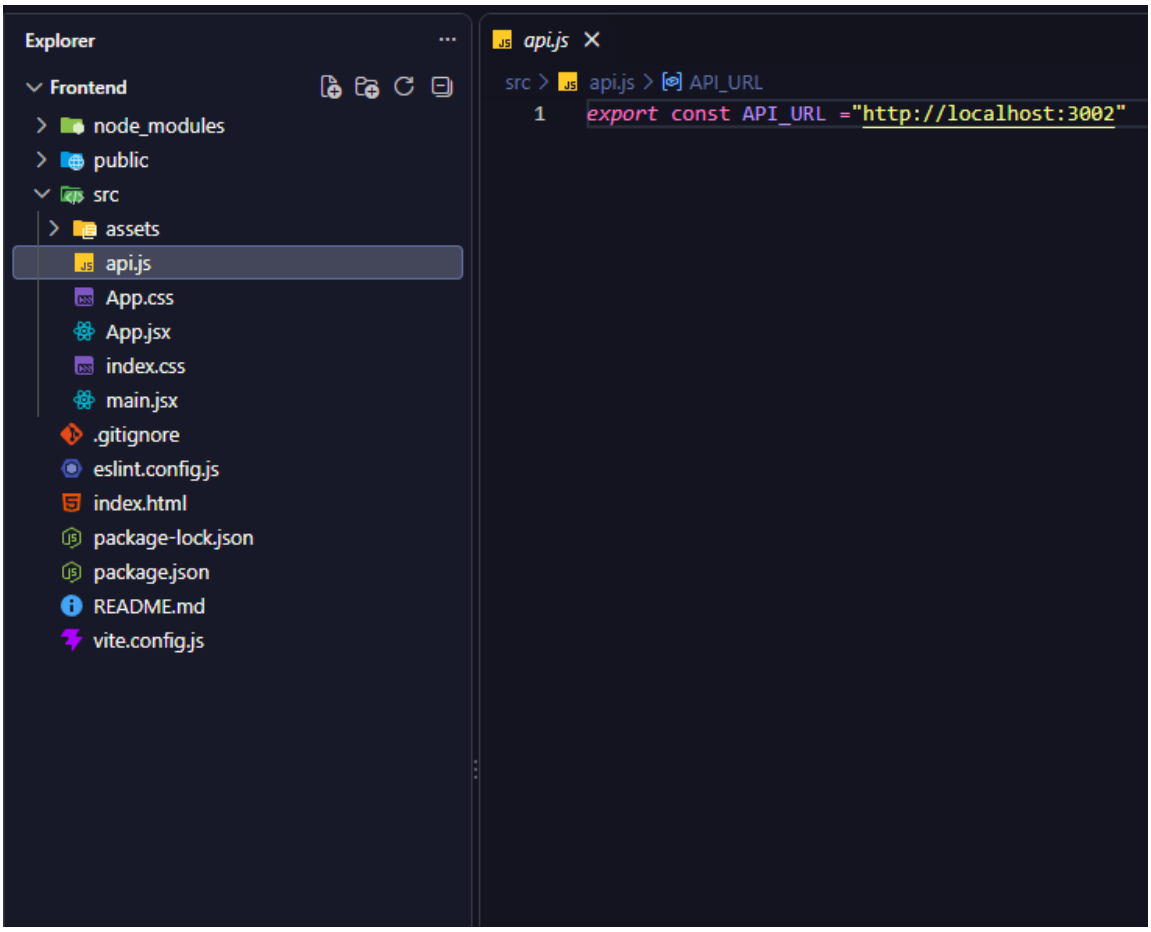


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1.2. Configuración de entorno



1.3. Creación de js con dirección URL de la API



1.4. Código Frontend

1.4.1. Componente Inicio.jsx

```
// Importar el useState
import { useState } from "react"
// Importar el API_URL
import {API_URL} from "../api"
// Importar componentes react Bootstrap
import { Container, Button } from "react-bootstrap"
// Creacion del componente
export default function Inicio(){
```

```
    //usestate
    const [mensaje,setMensaje]= useState("")
    //Creacion de la funcion consultarInicio
    async function consultarInicio() {
        const respuesta = await fetch(`${API_URL}/inicio`)
        const datos = await respuesta.json()
        setMensaje(datos.mensaje)
    }
    return(
        <Container>
            <h2>Inicio</h2>
            <Button onClick={consultarInicio}>Consultar Inicio</Button>
            <h1>{mensaje}</h1>
        </Container>
    )
}
```

1.4.2. Componente Informacion.jsx

```
// importar useState
import { useState } from "react"
// importar  el API
import { API_URL } from "../api"
// importar los componentes react bootstrap
import { Container, Button } from "react-bootstrap"
// importar el componente de las aletras sweetalert2
import Swal from "sweetalert2"
// Creacion del componente
export default function Informacion(){
    //useState
    const [info,setInfo] = useState(null)
    // funcion de consultar informacion
    async function consultarInformacion() {
        const respuesta= await fetch(`${API_URL}/informacion`)
        const datos = await respuesta.json()
        setInfo(datos)
    Swal.fire({
        title:datos.nombre,
        html:`
```

```
        <p><strong>Version: </strong>${datos.version}</p>
        <p><strong>Descripcion: </strong>${datos.descripcion}</p>
        `,
        icon: "info",
        confirmButtonText:"Entendido"
    })
    }
    return(
        <Container>
            <h2>Informacion de la aplicacion</h2>
            <Button onClick={consultarInformacion}>Info</Button>
            {info &&(
                <Container>
                    <p>Nombre: {info.nombre}</p>
                    <p>Version: {info.version}</p>
                    <p>Descripcion: {info.descripcion}</p>
                </Container>
            )}
        </Container>
    )
}
```

1.4.3. Componente Solicitudes.jsx

```
// importar useState
import { useState } from "react"
// importar  el API
import { API_URL } from "../api"
// importar los componentes react bootstrap
import { Container, Button, Form } from "react-bootstrap"
// importar el componente de las aletras sweetalert2
import Swal from "sweetalert2"
// Creacion del componente
export default function Solicitudes(){
    const [solicitud,setSolicitud] =useState({
        nombre:"",
        apellido:"",
        correo:""
    })
```

```
function manejarCambio(evento){
    const {name,value} = evento.target
    setSolicitud({
        ...solicitud,
        [name]:value
    })
}
async function enviarSolicitud(evento) {
    evento.preventDefault()
    const respuesta = await fetch(`${API_URL}/solicitudes`,{
        method:"POST",
        headers:{
            "conten-Type":"application/json"
        },
        body: JSON.stringify(solicitud)
    })
    const datos = await respuesta.json()
    Swal.fire({
        title:respuesta.ok ? "Solicitud recibida":"Error",
        text: datos.mensaje || datos.error,
        icon: respuesta.ok ? "success":"error",
        confirmButtonText:"Entendido"
    })
}
return(
    <Container>
        <h2>Enviar Solicitudes</h2>
        <Form onSubmit={enviarSolicitud}>
            <Form.Group className="mb-3">
            <Form.Label>Nombre</Form.Label>
            <Form.Control
                type="text"
                name="nombre"
                value={solicitud.nombre}
                onChange={manejarCambio}
                placeholder="Ingrese su nombre"
                required
            />
            </Form.Group>
```

```
                <Form.Group className="mb-3">
                <Form.Label>Apellido</Form.Label>
                <Form.Control
                    type="text"
                    name="apellido"
                    value={solicitud.apellido}
                    onChange={manejarCambio}
                    placeholder="Ingrese su apellido"
                    required
                />
                </Form.Group>

                <Form.Group className="mb-3">
                <Form.Label>Correo electrónico</Form.Label>
                <Form.Control
                    type="email"
                    name="correo"
                    value={solicitud.correo}
                    onChange={manejarCambio}
                    placeholder="Ingrese su correo"
                    required
                />
                </Form.Group>

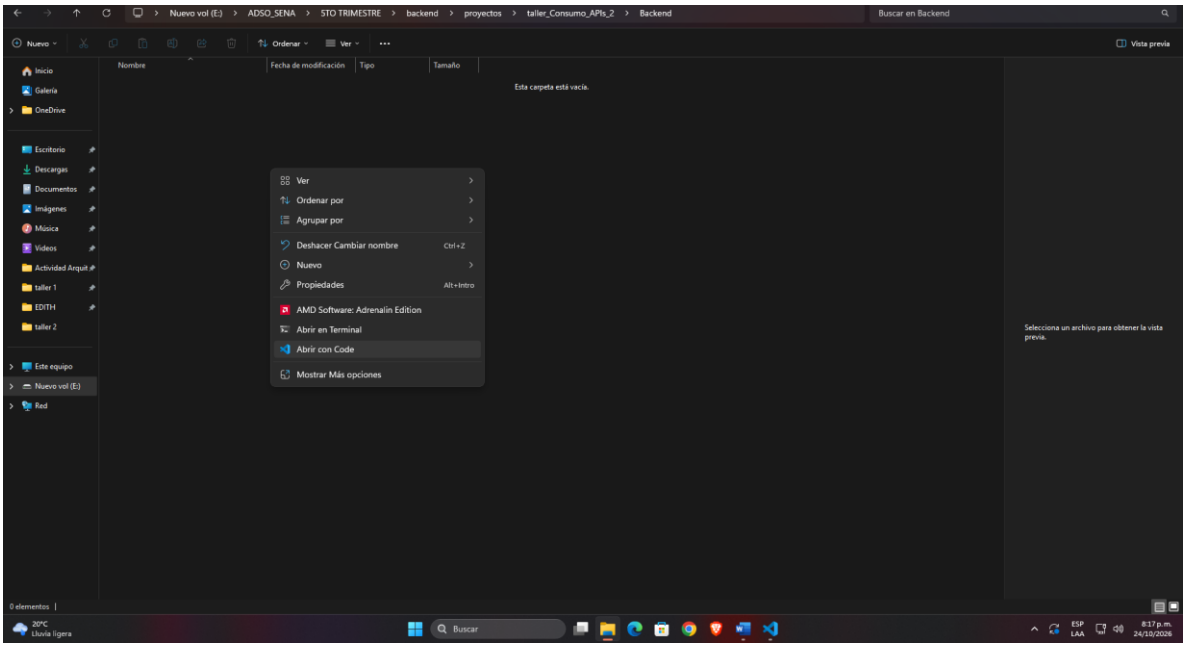
                <Button variant="primary" type="submit">
                Enviar Solicitud
                </Button>
            </Form>
        </Container>
    )
}
```

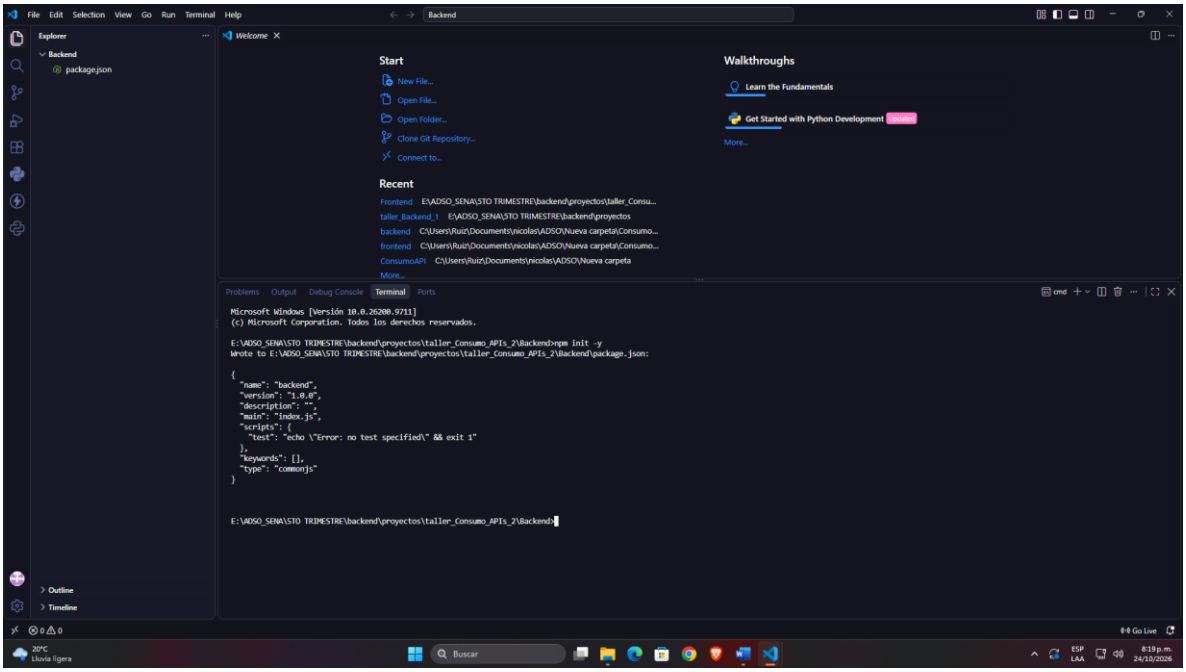
1.5. Evidencias

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2. Creación proyecto backend

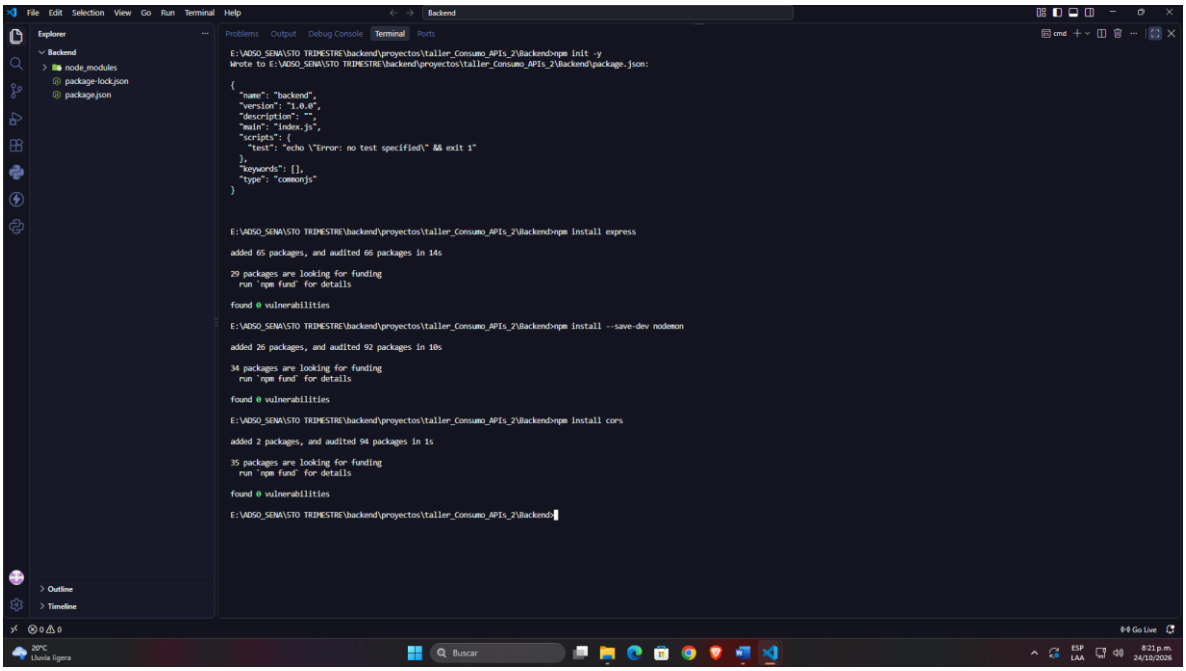
2.1. Creación de carpeta y abrirla con vscode

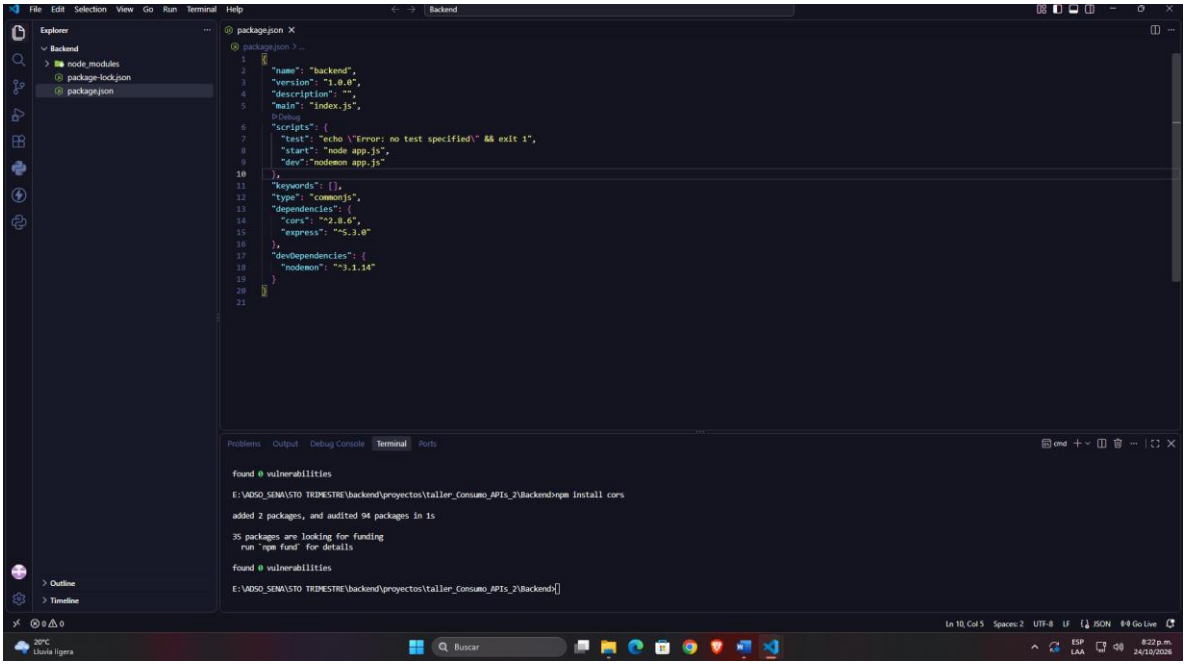




PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2.2. Configuración del entorno





2.3. Código backend

2.3.1. Código completo

```
// Inportar CORS
const cors = require('cors')
// Importar Express
const express = require('express')
// Crear aplicación
const app = express()
// Definir el puerto
const PORT = 3002
// Configurar los permisos CORS
app.use(cors({
    origin:"http://localhost:5173"
}))
// Configuración para permitir que express lea JSON
app.use(express.json())
//Registrar las peticiones que llegan
app.use((req,res,next)=>{
    console.log(req.method, req.url)
    next()
})
// Creación del endpoint GET /inicio
app.get('/inicio',(req,res)=>{
    res.json({
        mensaje:"Bienvenido a mi APP"
    })
})
// Creación del endpoint GET /informacion
app.get('/informacion',(req,res)=>{
    res.json({
        nombre:"App de prueba",
        version:"1.0.0",
        descripcion:"En esta app podremos demostrar el uso de las
conexiones de APIs Frontend con el Backend"
    })
})
// Creacion del endpoint POST /solicitudes
app.post('/solicitudes',(req,res)=>{
```

```
    // Extraer los datos del req.body
    const {nombre,apellido,correo}= req.body
    // Comprobacion de campos vacios
    if(!nombre.trim() || !apellido.trim() || !correo.trim()){
        return res.status(400).json({
            error:"Debe diligenciar todos los campos"
        })
    }
    // Validamos el formato del correo
    const regexCorreo = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
    // Comprobamos el formato del correo y generamos una respuesta
    if(!regexCorreo.test(correo)){
        return res.status(400).json({
            error:"El correo no tiene un formato valido"
        })
    }
    // Mensaje de comprobacion de la solicitud recibida
    res.status(201).json({
        mensaje:"Solicitud recibida"
    })
})
// Middleware por ruta inexistente
app.use((req,res)=>{
    res.status(404).json({
        error:"Ruta no encontrada"
    })
})
// Poner el servidor en funcionamiento
app.listen(PORT, ()=>{
    console.log(`Servidor ejecutandose en http://localhost:${PORT}`)
})
```

2.3.2. Explicación paso a paso

2.3.2.1. Importación y configuración de cors

```
const cors = require('cors')
```

```
app.use(cors({
    origin:"http://localhost:5173"
}))
```

2.3.2.2. Importación y configuración de express

```
const express = require('express')
```

```
app.use(express.json())
```

2.3.2.3. Crear aplicación

```
const app = express()
```

2.3.2.4. Definición del puerto

```
const PORT = 3002
```

2.3.2.5. Registro de peticiones que llegan

```
app.use((req,res,next)=>{
    console.log(req.method, req.url)
    next()
})
```

2.3.2.6. Creación de los endpoints

2.3.2.6.1. Endpoint GET /inicio

```
app.get('/inicio',(req,res)=>{
```

```
    res.json({
        mensaje:"Bienvenido a mi APP"
    })
})
```

app.get('/inicio',(req,res)=>{} → esta parte de la línea es la creación del middleware con el nombre de la petición en este caso se llama /inicio y recibirá 2 parametros (req,res); → res.json({ mensaje:" "}) esto es una estructura json con el mensaje que responde la petición.

2.3.2.6.2. Endpoint GET /información

```
app.get('/informacion',(req,res)=>{
    res.json({
        nombre:"App de prueba",
        version:"1.0.0",
        descripcion:"En esta app podremos demostrar el uso de las
conexiones de APIs Frontend con el Backend"
    })
})
```

app.get('/informacion',(req,res)=>{} → esta parte de la línea es la creación del middleware con el nombre de la petición en este caso se llama /informacion y recibirá 2 parametros (req,res); → res.json({}) es el objeto json que muestra la información o respuesta de la solicitud

2.3.2.6.3. Endpoint POST /solicitudes

```
app.post('/solicitudes',(req,res)=>{
const {nombre,apellido,correo}= req.body
```

```
if(!nombre.trim() || !apellido.trim() || !correo.trim()){
        return res.status(400).json({
            error:"Debe diligenciar todos los campos"
        })
    }
```

app.post('/solicitudes',(req,res)=>{} define la ruta del servidor express,

const {nombre,apellido,correo} = req.body determina los valores de la solicitud

if(¡nombre.trim() || ¡apellido.trim() || ¡correo.trim()){} en esta condición se comprueba si hay algún campo vacio

return res.status(400).json({ error: "mensaje error"})

```
 const regexCorreo = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
```

En esta línea se verifica o valida el formato del correo

```
if(!regexCorreo.test(correo)){
        return res.status(400).json({
            error:"El correo no tiene un formato valido"
        })
    }
```

En esta línea se valida el formato del correo y muestra el mensaje de error si no es válido y el estado de la petición también cambiaria

```
res.status(201).json({
        mensaje:"Solicitud recibida"
    })
```

Esta línea muestra que la solicitud es recibida

2.3.2.6.4. Middleware por ruta inexistente

```
app.use((req,res)=>{
    res.status(404).json({
        error:"Ruta no encontrada"
    })
})
```

2.3.2.6.5. Middleware para poner servidor en funcionamiento

```
app.listen(PORT, ()=>{
    console.log(`Servidor ejecutandose en http://localhost:${PORT}`)
})
```

3. Investigación sobre consumo de API

3.1. ¿Qué es una API y para que sirve en una aplicación web?

Una API (interfaz de programación de aplicaciones) es un conjunto de reglas y funciones que permite a un programa realizar peticiones u otro sin conocer su funcionamiento interno, es como un puente entre lo que ve el usuario y los datos o la lógica que viven en el servidor.

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction

3.2. ¿Qué diferencia existe entre frontend y backend?

El frontend es como la interfaz de usuario la cual el usuario interactua,

El backend se encarga del procesamiento de solicitudes, de aplicar las reglas y la comunicación entre la base de datos

3.3. ¿Qué significa consumir una API?

Consumir una API significa que un cliente haga peticiones y use sus respuestas

3.4. ¿Qué función cumple fetch()?

Recibir la ruta del recurso y opcionalmente un objeto de opciones, en los cuales se se pueden definir loa métodos POST, headers, body; esta devuelve una Promise; también es una forma de comunicarse con una API

https://docs.w3cub.com/dom/fetch_api

3.5. ¿Por qué se utilizan async y await en una petición HTTP?

Se utilizan para funciones asíncronas donde es async nos indica o permite declarar una función como asíncrona y el await permite esperar el resultado de una promesa dentro de esa función antes de continuar con la siguiente instrucción

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function

3.6. ¿Qué es una promesa en JavaScript?

Una promise es un objeto que representa el resultado futuro de una operación asíncrona, también permiten ejecutar acciones cuando una operación termina, manejar errores y organizar tareas que dependen de resultados obtenidos posteriormente

http://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise

3.7. ¿Para que sirve response.json()?

Este método sirve para leer el cuerpo de una respuesta HTTP e interpretar su contenido como JSON, este método también es asíncrono y se usa normalmente con un await, hay que tener cuidado al usarlo porque si el cuerpo no contiene un JSON valido podria fallar

https://developer.mozilla.org/en-US/docs/Web/API/Response/json

3.8. ¿Qué diferencia existe entre una petición GET y una petición POST?

GET: se usa para solicitar u obtener información de un servidor

POST: se usa para enviar datos al servidor para que sean procesados

https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/get-vs-post/

3.9. ¿Qué función cumple las cabezeras HTTP y por qué se utiliza Content-Type:application/json?

Las cabeceras o headers nos permiten enviar información adicional sobre una petición o una respuesta, como el tipo de contenido.

Content-Type : application/json indica que el cuerpo de la petición contiene datos en formato JSON, lo cual permite al servidor identificar el formato de la información que esta recibiendo.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch#setting_headers

3.10. ¿Por qué se utiliza JSON.stringify() al enviar datos en una petición POST?

Porque convierte un valor de JavaScript como un objeto en una cadena de texto con formato JSON

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

3.11. ¿Qué diferencia existe entre un error de conexión y una respuesta HTTP con código 400?

Un error de conexión ocurre cuando no se logra completar la comunicación con el servidor esto ocurre porque el backend esta apagado o la dirección es incorrecta.

El código HTTP 400 es cuando una solicitud falla o no fue posible procesarla

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/400

3.12. ¿Qué función cumplen useState y los eventos onClick y onSubmit?

useState: permite guardar y actualizar el estado de un componente

onClick: es un evento que permite ejecutar una función cuando el usuario hace clic en un elemento

onSubmit: es un evento que permite ejecutar una función cuando se envia un formulario

https://react.dev/reference/react/useState

https://react.dev/learn/responding-to-events

https://react.dev/reference/react-dom/components/form

3.13. ¿Qué es CORS y por qué puede afectar la comunicación entre React y Express?

CORS significa Cross-Origin-Resource-Sharing o intercambio de origen cruzado; es un mecanismo de seguridad de los navegadores que controla si una página web puede leer respuestas de un servidor ubicado en un origen diferente

https://expressjs.com/es/resources/middleware/cors/

3.14. ¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend?

Se puede provocar errores de validación, respuestas HTTP como 400, también depende de las validaciones que se haya implementado en el servidor.

https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

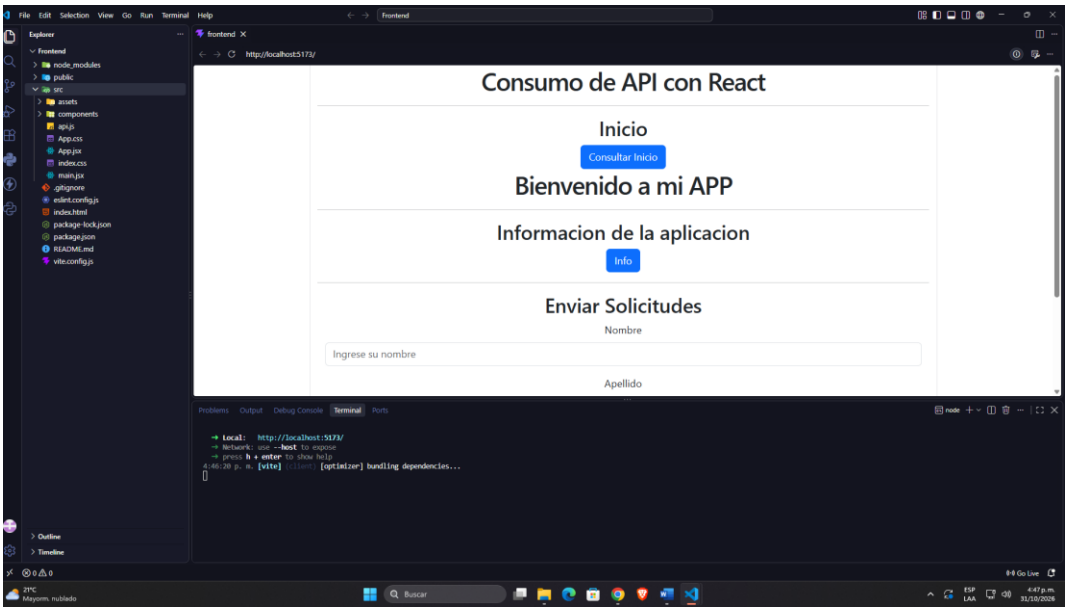
3.15. ¿Por qué no se debe considerar exitosa toda petición solamente porque fetch() devolvió una respuesta?

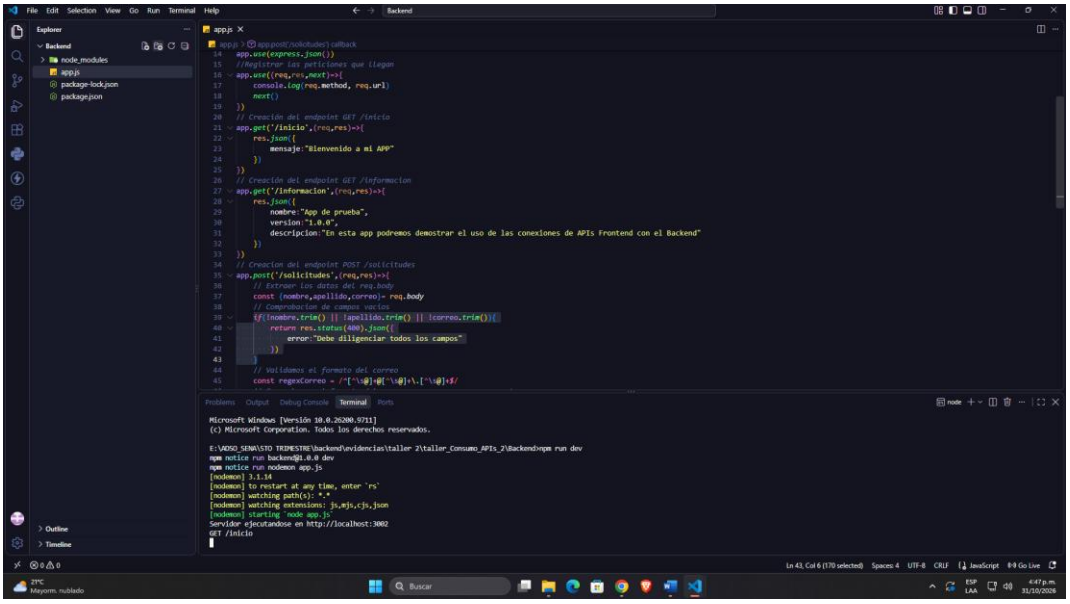
En muchas ocasiones el fetch() puede devolver una respuesta incluso cuando el servidor informa que ocurrió un error, esto significa que a pesar de recibir una respuesta no se garantiza que la operación se ha completado.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4. Evidencias de pruebas

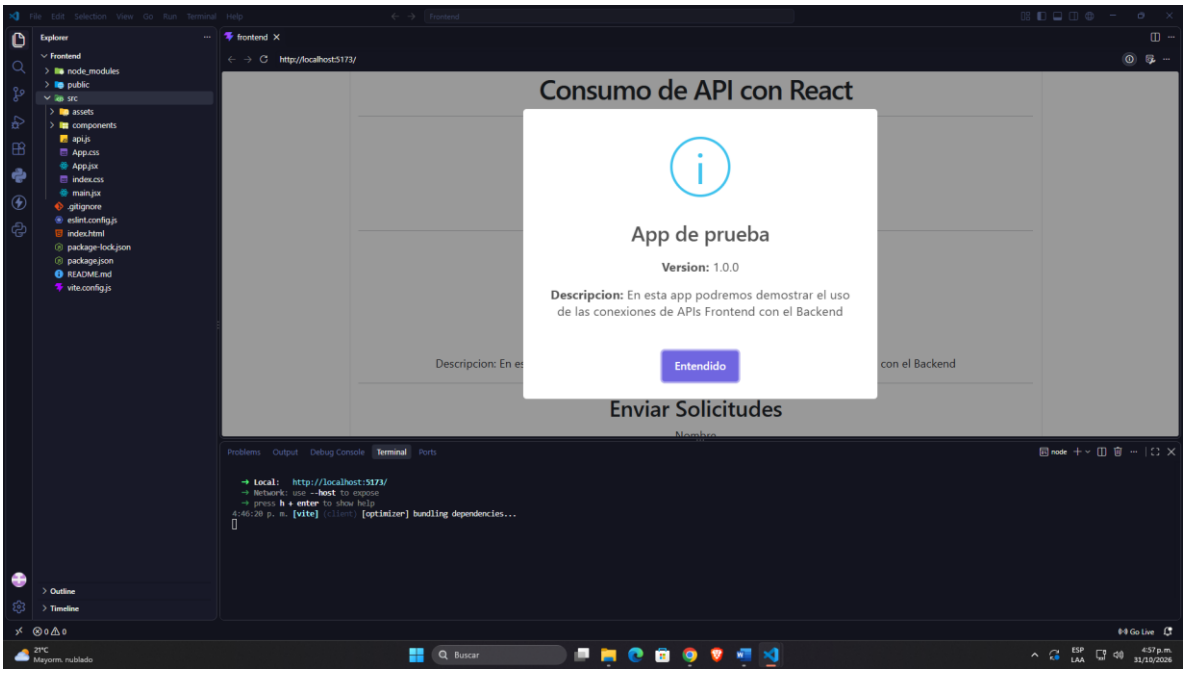
4.1. Consultar /inicio

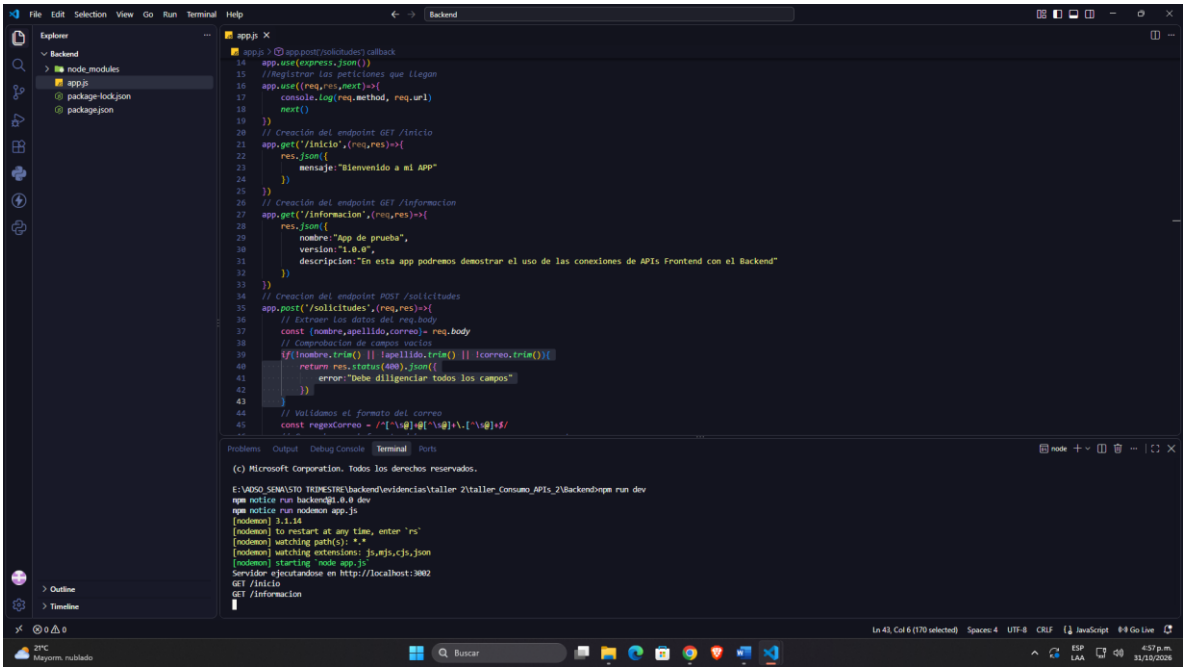




PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

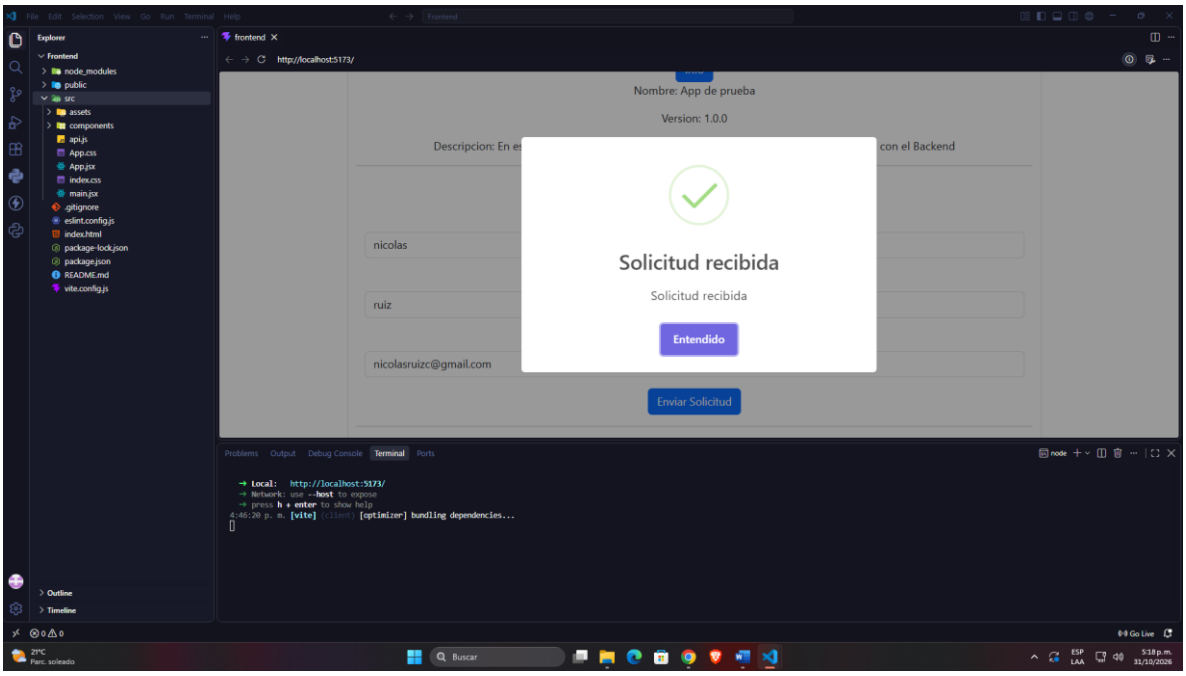
4.2. Consultar /información

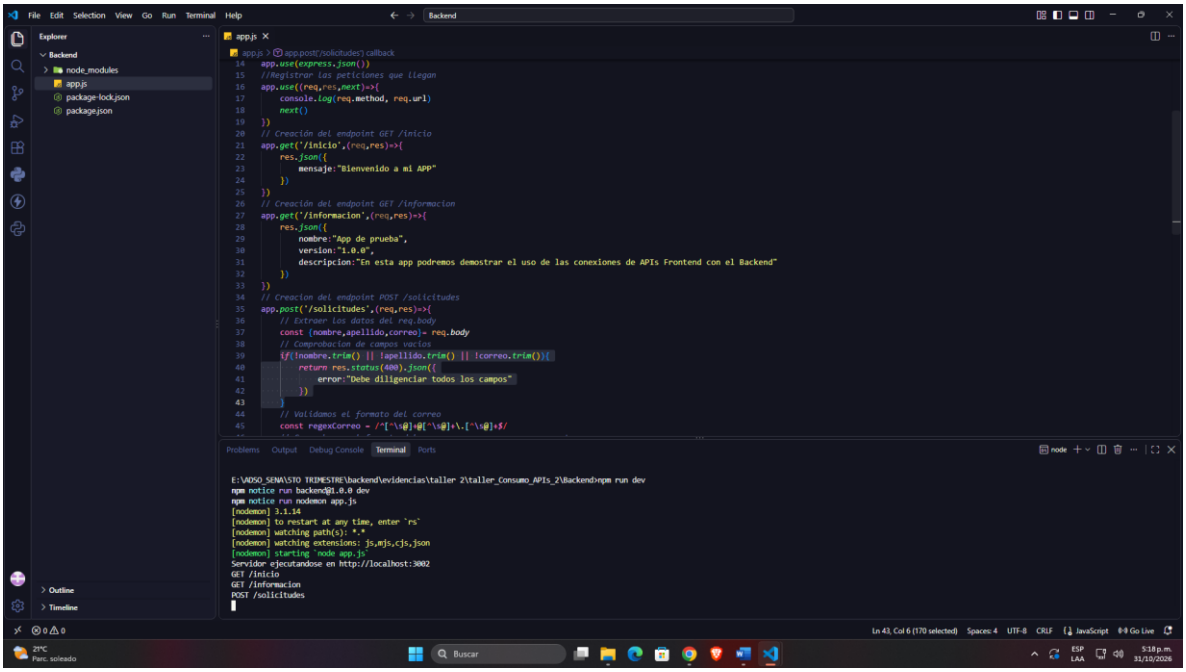




PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

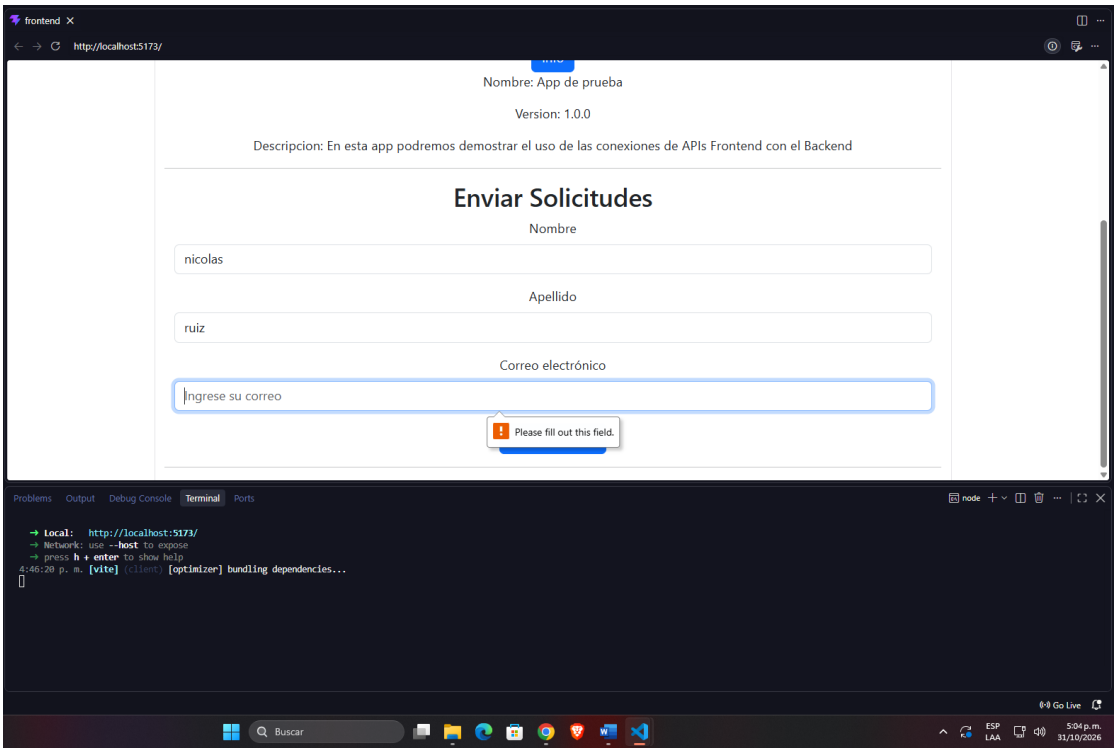
4.3. Enviar un formulario completo



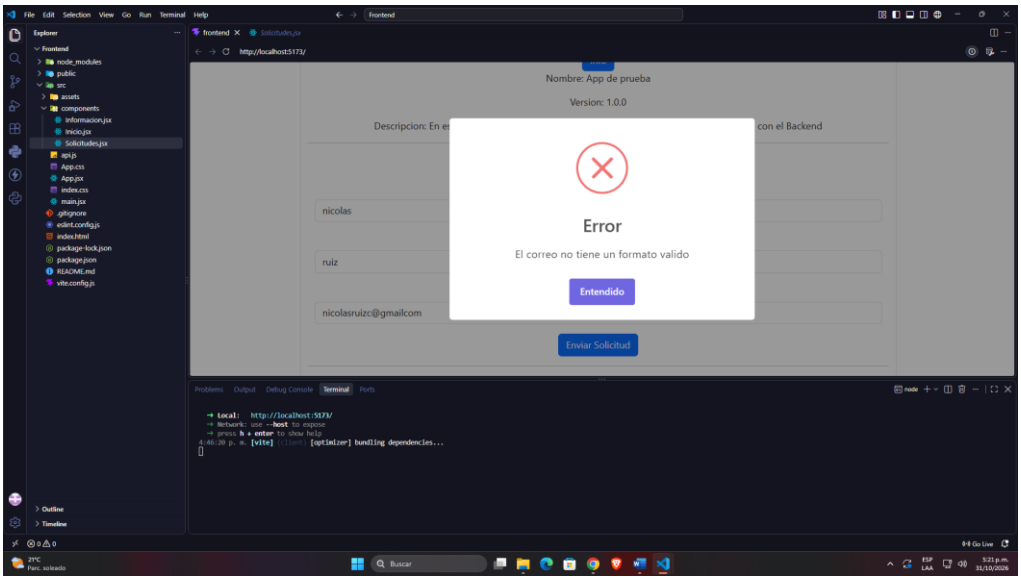


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4.4. Enviar campos vacíos



4.5. Enviar un correo invalido



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL

FORMATO ENTREGA DE EVIDENCIAS